

LC 2699: MODIFY GRAPH EDGE WEIGHTS

© Authored by: Arpit Raj, LNMIIT Jaipur
Date: July 24, 2026

Problem Overview & Basic Setup

Example Graph parameters:

- *start = 0*
- *end = 2*
- *target = 6*

we can only change edges with cost = -1
for now cost $\rightarrow 4 + 3 = 7$ (Cant travel path with -1 edge till they are > 0)

Goal: we need 6

- If we change -1 to +1 then $5 + 1 = 6$
- we need to return a vector of $\{u, v, w\}$
- if possible return $\{\}$ changing one edge change shortest path of the intire graph

Key Observations

Obv-1 $\rightarrow$ ignore all -1 edges

only fixed edges remain

suppose shortest $>$ target

then return $\{\}$ as we can only increase shortest not decrease

Ob-2

shortest $==$ target

then simply make -1 $\rightarrow \infty$ so that they never affect shortest path

Greedy Idea

if $\text{shortest} < \text{target}$

- then, simply turn every -1 to 1 which gives the smallest possible shortest path
- now run Dijkstras again

If still $\text{shortest} < \text{target}$:

then simply add the difference to any one edge because every other -1 is already 1 (the minimum)

Algorithm Flow

ignore -1, run Dijkstras

- $\text{dist} > \text{target} \rightarrow \text{return } \{\}$
(because it's optimal and turning -1 to 1 wont help as it will only give a distance more than it)
- $\text{dist} == \text{target} \rightarrow \text{make } -1 \rightarrow \infty$
- $\text{dist} < \text{target} \rightarrow \text{make all } -1 \rightarrow 1 \rightarrow \text{run Dijkstras} \rightarrow \text{if } \text{shortest} \leq \text{target} \text{ increase that edge by } (\text{target} - \text{currdist}) \text{ and remaining } -1 \rightarrow \text{infinity}$

Implementation Code

```
class Solution {
public:

    long long djik(int n, vector<vector<int>>& edges, int source, int
destination){

        vector<vector<pair<int,long long>>> adj(n);
        for(auto &it:edges){

            int u=it[0];
            int v=it[1];
            int w=it[2];
            if(w==-1) continue;

            adj[u].push_back({v,w});
            adj[v].push_back({u,w});

        }
        priority_queue<pair<long long,int>,vector<pair<long
long,int>>,greater<pair<long long,int>>> pq;
        vector<long long> dist(n,LLONG_MAX);

        dist[source]=0;
        pq.push({0,source});

        while(!pq.empty()){
            int currnode=pq.top().second;
            long long currdiss=pq.top().first;
            pq.pop();

            if(currdiss>dist[currnode]) continue;
            if(currnode==destination) return currdiss;

            for(auto &it:adj[currnode]){

                int newnode=it.first;
                long long cost=it.second;
                long long newdiss=currdiss+cost;
                if(newdiss<dist[newnode]){
                    dist[newnode]=newdiss;
                    pq.push({newdiss,newnode});
                }

            }

        }

        return dist[destination];

    }

    vector<vector<int>> modifiedGraphEdges(int n, vector<vector<int>>& edges,
int source, int destination, int target) {
```

```

        long long dist=djk(n,edges,source,destination); //bina kisi edge change
        ke check karo
        if(dist<target){
            return {}; //because already sabse chhota path he , to weight change
            wont help
            //jaayega to abhi bhi ussi path se
        }

        if(dist==target){ //matlab already optimal answer he , to saare-1 ko
        infinite kr do
            //so that they dont interfere with this answer
            for(auto& it:edges){
                if(it[2]==-1){
                    it[2]=1e9;
                }
            }

            return edges;
        }

        else{ //agar dist>target he to ab ham graph weight adjust kr skte in
        hope of finding
            //even shorter distance to destination
            //ek ek karke -1 ko 1 bnayenge and agar dist<=target aaya to utna
            hi difference
            //uss edge me daal denge to match the total cost with target
            //all other edge can be infinite so that they dont interfere

            for(auto &it:edges){

                if(it[2]==-1){
                    it[2]=1;
                    dist=djk(n,edges,source,destination);

                    if(dist<=target){
                        it[2]+=(target-dist);

                        for(auto &ed:edges){
                            if(ed[2]==-1){
                                ed[2]=1e9;
                            }
                        }

                        return edges;
                    }
                }
            }

        }

        return {};
    }
};

```